

DOM Lifecycle Audit

Denai Clinical Platform — Deterministic Rendering Preservation Document

Audit Scope and Method

This audit maps DOM ownership, lifecycle behavior, structural rebuild behavior, render patching behavior, async DOM mutation behavior, and state-node risk zones for every major UI region. All findings reflect the existing execution model exactly. No optimizations, refactors, or redesigns are proposed. All ordering contracts, overwrite sequences, rebuild dependencies, and async lifecycle interactions are preserved as documented.

1. AI Card System

DOM Ownership

Primary owner: `renderMainPanels` → `withErrorBoundary` closure → `buildAICardStructure` / `updateAICard` / `updateAICardMulti` **Secondary owner (preview):** `updatePreview` →

`buildAICardStructure` / `updateAICard` / `updateAICardMulti` (direct, not via `withErrorBoundary`)

Shared ownership: `showSkeleton('aiCardBody')` in `switchPatient` temporarily seizes ownership of `#aiCardBody` and invalidates the built-state flag.

Creation Path

- `buildAICardStructure()` checks `body.dataset.built === '1'`
- If NOT set: full `innerHTML` replace of `#aiCardBody` with the full card DOM tree
- Sets `body.dataset.built = '1'` on completion
- `showSkeleton('aiCardBody')` clears `#aiCardBody` via `innerHTML` replace and does NOT set `dataset.built` — triggering rebuild on the next render

Update (Patch) Path

- `buildAICardStructure()` finds `dataset.built === '1'` → **NO-OP** (skips `innerHTML` replace)
- `updateAICard(ai)` or `updateAICardMulti(ai)` runs patch operations:
 - Ring SVG `stroke-dashoffset` attribute writes

- `animateNumber('successVal', ...)` DOM value writes
- `#caseClassBadge` visibility toggle + `textContent` write
- `#recTopReason` show/hide
- `typewriterEffect(recText, recStr, 30)` → async character-by-character append

Destruction Path

- No explicit destruction. Card is replaced in two scenarios:
 1. `showSkeleton('aiCardBody')` — full `innerHTML` overwrite with skeleton markup
 2. `withErrorBoundary` catch — `innerHTML` overwrite with error fallback card

Rebuild Triggers

- `switchPatient()` → `showSkeleton('aiCardBody')` → clears `dataset.built` → next `render(S)` forces full rebuild
- `withErrorBoundary` error catch → error fallback replaces `#aiCardBody` → `dataset.built` is lost → subsequent renders attempt to patch elements that no longer exist

Conditional Rendering Behavior

- `ai.isMultiTooth` → routes to `updateAICardMulti(ai)` instead of `updateAICard(ai)`
- Both `render(S)` path and `updatePreview()` path evaluate this branch
- `withErrorBoundary` on `aiCardBody` catches structural failures and replaces with fallback

Dataset Flag Usage

- `body.dataset.built === '1'` : The single gate controlling structural rebuild vs. patch
- Set by `buildAICardStructure()` on successful build
- Cleared implicitly by `showSkeleton()` (`innerHTML` overwrite discards the element carrying the flag)
- NOT set by the `withErrorBoundary` error fallback → fallback has no `dataset.built` → subsequent render calls attempt patch on error markup

innerHTML Replacement Usage

- **Full replacement:** `buildAICardStructure()` when `dataset.built` is absent
- **Full replacement:** `showSkeleton('aiCardBody')` in `switchPatient()`
- **Full replacement:** `withErrorBoundary` catch block
- **Partial writes:** `updateAICard()` / `updateAICardMulti()` write to child element IDs via `$()` selectors — these are patch writes, NOT `innerHTML` replacements of `#aiCardBody`

Stale Node Risks

- If `withErrorBoundary` catch fires and replaces `#aiCardBody` with error fallback markup, all subsequent `$()` lookups for child IDs (`recText` , `successVal` , `caseClassBadge` , `recTopReason`) return `null`
- `updateAICard()` calls on null nodes are **silent no-ops** — no exception thrown
- The error fallback persists indefinitely until `buildAICardStructure(true)` force-rebuild, which is never automatically triggered after an error

Orphan Listener Risks

- `typewriterEffect` uses a module-scoped `setInterval ( _typewriterTimer )`
- The interval appends characters to `recText` element via DOM reference captured at call time
- If `showSkeleton()` or error fallback replaces `#aiCardBody` while `_typewriterTimer` is mid-run, the interval continues firing against the **destroyed element reference**
- The interval is only cleared by the next `clearInterval(_typewriterTimer)` call at the top of the next `typewriterEffect()` invocation

Async Overwrite Risks

- `typewriterEffect ( _typewriterTimer , 30ms interval )`: ongoing after `renderMainPanels` returns
- A subsequent `render(S)` (e.g., from the 200ms save debounce triggering a path that calls `render`) cancels and restarts the typewriter from empty — mid-animation text is destroyed
- `updatePreview()` also calls `typewriterEffect` directly, canceling any in-progress `render(S)` typewriter animation

Shared DOM Ownership

- `render(S)` path and `updatePreview()` path both write to `#aiCardBody` children
- `switchPatient()` writes to `#aiCardBody` via `showSkeleton()`
- No coordination lock exists between these owners

Render Recovery Behavior

- `withErrorBoundary` catch: replaces container with error fallback card
- Recovery from error state requires a `buildAICardStructure(true)` force call — this is NOT automatically invoked in any current code path after a catch
- Error state is therefore terminal until the next `switchPatient()` (which calls `showSkeleton()` , clearing the error fallback and resetting `dataset.built`)

Fallback Replacement Behavior

- Error fallback: full `innerHTML` of `#aiCardBody` replaced with an error card HTML string
- Skeleton fallback: full `innerHTML` of `#aiCardBody` replaced with skeleton markup; sets `aria-busy='true'`

Lazy Rendering Behavior

- None. The AI card always renders synchronously within its execution path, whether via `render(S)` or `updatePreview()`

Observer-Controlled Rendering

- None directly. The AI card is not observer-gated.

Timer-Controlled Rendering

- `typewriterEffect (_typewriterTimer): 30ms setInterval` — ongoing character append to `recText`
- This timer is NOT in `CleanupRegistry`; it is managed only by `clearInterval` at the start of the next `typewriterEffect` call

Visibility-Controlled Rendering

- None. The AI card always renders regardless of scroll position.

Structural vs. Patch Rendering Distinction

- **Structural:** `buildAICardStructure()` — full `innerHTML` rebuild when `dataset.built` is absent
- **Patch:** `updateAICard()` / `updateAICardMulti()` — attribute writes and child element text mutations when `dataset.built === '1'`
- The `dataset.built` flag is the sole determinant of which mode executes

2. Comparison Table System

DOM Ownership

Primary owner: `lazyRenderComparisonTable(state, ai)` called from `renderMainPanels` **Secondary owner (preview):** `renderComparisonTable(tempState, ai)` called directly from `updatePreview()` — bypasses the lazy mechanism entirely **Tertiary owner:** `toggleMultiSite()` — executes a direct pre-wipe of `#comparisonTableBody`

Creation Path

- `renderComparisonTable(state, ai) : full innerHTML write to #comparisonTableBody`
- Creates all rows, headers, and cell content as a single HTML string assignment

Update Path

- Same as creation: full `innerHTML` replacement of `#comparisonTableBody`
- No patching. Every render is a structural rebuild of the entire table body

Destruction Path

- `toggleMultiSite() : _tbody.innerHTML = ''` — explicit wipe before `render(S)` is called
- This pre-wipe is unconditional regardless of table visibility or scroll position

Rebuild Triggers

- Any `render(S)` call → `lazyRenderComparisonTable` → rebuild (if visible) or observer-deferred rebuild (if offscreen)
- `updatePreview()` → `renderComparisonTable` direct call → always immediate rebuild
- `toggleMultiSite()` → pre-wipe → `render(S)` → rebuild sequence
- `selectTx()` → `render(S)` → rebuild (column highlight changes with `state.tx`)

Conditional Rendering Behavior

- `lazyRenderComparisonTable` checks element visibility at call time:
 - **Visible path:** calls `renderComparisonTable(state, ai)` synchronously
 - **Offscreen path:** disconnects prior `_compTableObserver`, creates new `IntersectionObserver` → defers `renderComparisonTable` until element scrolls into view
- `updatePreview()` always calls `renderComparisonTable` directly — never uses the lazy path

Dataset Flag Usage

- None on the comparison table itself

innerHTML Replacement Usage

- `#comparisonTableBody`: full `innerHTML` replacement on every render (no patching)
- `toggleMultiSite()` pre-wipe: `innerHTML = ''` directly on `_tbody`

Stale Node Risks

- **Observer closure trap:** `_compTableObserver` captures `state` and `ai` via closure at creation time inside `lazyRenderComparisonTable`. If `s` mutates between observer creation and observer fire (user scrolls table into view after a state change), the observer fires `renderComparisonTable` with the **stale closure's `state` / `ai`** — not the current `s`
- The comparison table then shows data for a historical state while all other panels show current state

Orphan Listener Risks

- `_compTableObserver` : the observer is disconnected on every new render call via `_compTableObserver.disconnect()` . However, if the observer fires its callback **between** the `disconnect()` call and the new observer setup (within the same synchronous block — not possible in practice), there is no risk window. The real risk is the closure state mismatch, not a listener leak.

Async Overwrite Risks

- Observer-deferred render: fires at an uncontrolled future time (user scroll event)
- If `render(S)` is called after the observer is set but before it fires, the new `render(S)` call disconnects the old observer and creates a new one. The old observer callback **never fires**
- If the observer fires during `updatePreview()` execution (multi-threaded environment, not applicable in browser JS), no conflict — but the observer's state `state` would still apply

Shared DOM Ownership

- `render(S)` path (via `lazyRenderComparisonTable`)
- `updatePreview()` path (via direct `renderComparisonTable`)
- `toggleMultiSite()` (via direct `_tbody.innerHTML = ''`)
- All three write to `#comparisonTableBody` with no coordination

Render Recovery Behavior

- `withErrorBoundary` on `compInlineTable` (the comparison inline cells, not the full table): replaces container with error card on failure
- The full `#comparisonTableBody` is NOT wrapped in `withErrorBoundary` ; rendering failures propagate uncaught

Fallback Replacement Behavior

- `withErrorBoundary` on `compInlineTable` : error card innerHTML replacement
- No fallback for `#comparisonTableBody` itself

Lazy Rendering Behavior

- `lazyRenderComparisonTable` : defers render until `IntersectionObserver` fires when element is scrolled into view
- Observer is stored in module-scoped `_compTableObserver` — disconnected and replaced on every `render(S)` call
- `updatePreview()` bypasses this entirely — always renders synchronously

Observer-Controlled Rendering

- `_compTableObserver ( IntersectionObserver )`: fires `renderComparisonTable(state, ai)` when `#comparisonTableBody` becomes visible
- Observer is created inside `lazyRenderComparisonTable` with `state` and `ai` captured by closure at creation time
- Closure values do NOT update if `s` mutates after observer creation

Timer-Controlled Rendering

- None directly. The comparison table render is either synchronous or observer-deferred.

Visibility-Controlled Rendering

- Yes: `lazyRenderComparisonTable` gates synchronous render on element visibility
- Invisible → observer-deferred; visible → synchronous

Structural vs. Patch Rendering Distinction

- **Structural only.** Every render is a full `innerHTML` replacement of `#comparisonTableBody`. No patching mechanism exists.

3. Compound / Multi-Site Rendering

DOM Ownership

Primary owner: `render(S)` compound branch → `renderCompoundSummary(compound, state)` for `#compoundSummaryBody` **Secondary owner:** `toggleMultiSite()` for compound banner (`compoundBanner.style.display`) and pre-wipe of `#comparisonTableBody` **SVG owner:** Two-pass pattern — `renderMainPanels` → `updateToothHighlight(effectiveState.tooth)` (first pass), then `render()` directly → `updateToothHighlight(primaryTooth)` + `updateToothHighlight2(secondaryTooth)` (second pass)

Creation Path

- `renderCompoundSummary(compound, state) : innerHTML` write to `#compoundSummaryBody`
- Compound banner visibility: `compoundBanner.style.display = 'flex'` in `toggleMultiSite(true)`
- Content populated only after `render(S) → renderCompoundSummary()`

Update Path

- Every `render(S)` in compound mode rewrites `#compoundSummaryBody` via `renderCompoundSummary`
- No patch path — full `innerHTML` replacement each render

Destruction Path

- `toggleMultiSite(false) : compoundBanner.style.display = 'none'`
- `switchPatient() : compoundBanner.style.display = 'none' (unconditional)`

Rebuild Triggers

- Any `render(S)` call while `S.multiSite && S.site2Tooth` is truthy
- `toggleMultiSite(true) → render(S) → compound branch fires`

Conditional Rendering Behavior

- `render(S)` branches on `S.multiSite && S.site2Tooth`:
 - **Compound branch:** `ClinicalEngine.processCompound(S) → renderCompoundSummary() → derive effectiveState → renderMainPanels(effectiveState, ai)`
 - **Default branch:** `ClinicalEngine.process(S) → renderMainPanels(state, ai)`
- If `S.multiSite` is true but `S.site2Tooth` is falsy: compound branch fires BUT `processCompound` defaults `site2Tooth` to `'#11'` — computation proceeds with a synthetic default

Dataset Flag Usage

- None on compound-specific elements

innerHTML Replacement Usage

- `#compoundSummaryBody` : full innerHTML on every compound render
- `compoundBanner` : display style mutation only (no innerHTML)

Stale Node Risks

- Compound banner visible-before-content window: `compoundBanner.style.display = 'flex'` fires before `renderCompoundSummary()` executes. The banner is visible with empty `#compoundSummaryBody` during the JavaScript execution gap between these two lines. Sub-millisecond but a real DOM state.
- `updateSiteTabUI()` NOT called on `toggleMultiSite(true)` : site tab labels retain static HTML values until `switchSite()` is explicitly called by the user. Tabs show stale labels (e.g., “Site 1” without tooth numbers) from enable until first tab click.

Orphan Listener Risks

- No compound-specific listeners. The compound SVG two-pass writes carry the same orphan risks as the SVG tooth highlighting system (see Section 9).

Async Overwrite Risks

- SVG two-pass overwrite: the second pass inside `render()` fires synchronously after `renderMainPanels()` returns and overwrites the first pass SVG state. This is intentional and load-

bearing.

- If `renderMainPanels` is ever called independently (e.g., extracted), the second-pass overwrite from `render()` will not execute — compound SVG state will be incorrect.

Shared DOM Ownership

- `#compoundSummaryBody`: shared between `renderCompoundSummary()` (in render path) and `toggleMultiSite()` (pre-wipe via `#comparisonTableBody`)
- Compound banner: written by `toggleMultiSite()` and `switchPatient()`
- SVG elements: written by both passes inside `render()` + `renderMainPanels()`

Render Recovery Behavior

- No `withErrorBoundary` wrapping `renderCompoundSummary`. Failures propagate uncaught.
- If `ClinicalEngine.processCompound(S)` returns falsy: `generateReport()` checks this and shows a toast + restores button. `render(S)` does NOT check for falsy compound result — it proceeds to `renderCompoundSummary` regardless.

Fallback Replacement Behavior

- None specific to compound rendering

Lazy Rendering Behavior

- None for compound summary. `renderCompoundSummary` always fires synchronously within `render(S)`.

Observer-Controlled Rendering

- None directly for compound summary. Comparison table inside compound render still uses `lazyRenderComparisonTable`.

Timer-Controlled Rendering

- None specific to compound path beyond what is inherited from `renderMainPanels` (typewriter, material fade).

Visibility-Controlled Rendering

- None for compound summary itself.

Structural vs. Patch Rendering Distinction

- **Structural only.** `#compoundSummaryBody` is always fully replaced via `innerHTML`.

4. Sidebar Patient List

DOM Ownership

Primary owner: `renderPatientList()` — writes `#patientList` innerHTML **Secondary trigger:** `saveState()` 200ms debounce timer → `renderPatientList()` as a hidden side effect **Tertiary trigger:** `switchPatient()` → synchronous `renderPatientList()` call before the 300ms deferred `render(s)`

Creation Path

- `renderPatientList()` : full innerHTML replacement of `#patientList` — generates all patient list items from `loadAllPatients()` (localStorage read)
- Reads from localStorage, NOT from `s` — sidebar reflects persisted state, not runtime state

Update Path

- Same as creation: full innerHTML replacement of `#patientList` on every call
- No patching. Every update is a full rebuild from localStorage.

Destruction Path

- No explicit destruction. Full replacement constitutes implicit destruction of prior items.

Rebuild Triggers

- `saveState()` debounce fires (200ms after last state change) → always triggers `renderPatientList()`
- `switchPatient()` → explicit synchronous call
- `addPatient()` → `renderPatientList()`
- `deletePatient()` → `renderPatientList()`
- Notes autosave (800ms debounce → `saveState()` → 200ms debounce → `renderPatientList()`)

Conditional Rendering Behavior

- Each patient item rendered with `_quickScore(p)` — lightweight per-patient score computed from list data, not from `s`
- Active patient highlighted by matching `p.id === activeId` (read from localStorage via `getActivePatientId()`)

Dataset Flag Usage

- None

innerHTML Replacement Usage

- `#patientList` : full innerHTML replacement on every `renderPatientList()` call

Stale Node Risks

- Sidebar reads from localStorage at render time. If `saveState()` has not yet fired (within the 200ms debounce window), the sidebar may show stale persisted data even though `s` has been updated. This is the intended behavior but creates a display lag of up to 200ms after any state change.
- During `switchPatient() : renderPatientList()` fires synchronously BEFORE the 300ms deferred `render(s)`. The sidebar shows the new active patient BEFORE the main panels update. This is the intended behavior but contributes to the 300ms mixed DOM state window.

Orphan Listener Risks

- Event delegation used for patient list clicks — listeners on the container `#patientList` survive the innerHTML rebuild
- Individual patient item elements are destroyed and recreated on every rebuild — any listeners directly bound to item elements (not delegation-based) would be orphaned. Current architecture uses event delegation, so this is not active but must be preserved.

Async Overwrite Risks

- `renderPatientList()` fires 200ms after ANY `saveState()` call — completely detached from user interaction
- This background render can fire during any other user action, replacing `#patientList` innerHTML mid-interaction
- Not wrapped in `withErrorBoundary` — failures propagate uncaught to the `setTimeout` callback and are silently swallowed by the browser

Shared DOM Ownership

- `renderPatientList()` is the sole writer to `#patientList`
- But it is triggered from multiple distinct orchestration paths with no coordination

Render Recovery Behavior

- None. No error boundary. Failure is silent.

Fallback Replacement Behavior

- None

Lazy Rendering Behavior

- None. `renderPatientList()` always executes fully when called.

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- Indirectly: triggered by the 200ms `saveState()` debounce timer, making it effectively a timer-controlled background render
- Also triggered by the 800ms notes autosave debounce chain (800ms notes → `saveState()` → 200ms → `renderPatientList()`)

Visibility-Controlled Rendering

- None

Structural vs. Patch Rendering Distinction

- **Structural only.** Always full innerHTML replacement. No patch path exists.
-

5. Preview Banner System

DOM Ownership

Primary owner: `updatePreview()` — sets `previewModeBanner.style.display = 'flex'` **Clearing owner:** `whatIfApply` click handler — sets `previewModeBanner.style.display = 'none'` **Missing clearing owner:** `switchPatient()` — does NOT reset banner visibility

Creation Path

- Banner markup is static HTML — always present in DOM
- Visibility toggled via `style.display` — not created or destroyed

Update Path

- `style.display = 'flex'` when `updatePreview()` completes
- `style.display = 'none'` when `whatIfApply` click handler fires after apply

Destruction Path

- None. Element is never removed from DOM.
- Display toggled to `none` on apply. NOT toggled on `switchPatient()` .

Rebuild Triggers

- N/A — static element, visibility-toggled only

Conditional Rendering Behavior

- `_isPreviewMode` flag is set to `true` inside `updatePreview()`
- `_isPreviewMode` is set to `false` inside the `whatIfApply` handler

- `_isPreviewMode` is NOT read by `render(s)` — the flag has no effect on main render path behavior
- The banner display does not gate any other render logic

Dataset Flag Usage

- None

innerHTML Replacement Usage

- None. Banner text content is static. No innerHTML writes after initial load.

Stale Node Risks

- **Patient switch stale banner:** `switchPatient()` does not reset `previewModeBanner.style.display`. If a patient was in preview mode, the banner persists on the new patient's view. The banner text references the prior patient's preview context. `_isPreviewMode` is also NOT reset in `switchPatient()`.
- Report generation does not check `_isPreviewMode`: if banner is visible (preview mode active), the report is generated from `s` (committed state), not `tempState`. No warning is displayed to the user about this mismatch.

Orphan Listener Risks

- None. Banner is display-toggled, not rebuilt.

Async Overwrite Risks

- None on the banner element itself. The stale-after-switch risk is a visibility persistence issue, not an async overwrite.

Shared DOM Ownership

- `updatePreview()` sets display to flex; `whatIfApply` sets to none; `switchPatient()` does not touch it

Render Recovery Behavior

- N/A

Fallback Replacement Behavior

- N/A

Lazy Rendering Behavior

- N/A

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- None. Display toggled synchronously.

Visibility-Controlled Rendering

- Self-referential: the element IS a visibility indicator; no external visibility controls gate it

Structural vs. Patch Rendering Distinction

- N/A — static element, display-toggled only
-

6. History Panel

DOM Ownership

Primary owner: `History.render()` — writes `#historyList` innerHTML **Trigger sites:**

`switchPatient()` (synchronous, pre-deferred-render), `selectTx()`, `applyAIRec()`, `whatIfApply`,
`generateReport()`, `addPatient()`, `saveEdit()`, `deletePatient()`

Creation Path

- `History.render()` : reads history entries for `s.id` from `localStorage` via `History.load(S.id)`, builds HTML string, writes to `#historyList` innerHTML

Update Path

- Same as creation: full innerHTML replacement of `#historyList` on every `History.render()` call
- `History.add(label, detail)` → `History.load(S.id)` → push entry → `History.save(list, S.id)` → `History.render()`

Destruction Path

- No explicit destruction. Full innerHTML replacement constitutes implicit destruction of prior entries.

Rebuild Triggers

- Every `History.add()` call triggers `History.render()`
- `switchPatient()` explicitly calls `History.render()` with the new `s.id` before the 300ms deferred render — history shows correct new patient data during the mixed DOM state window

Conditional Rendering Behavior

- History is keyed by `s.id` — `History.load(S.id)` reads the current patient's history entries
- If called during the 300ms `switchPatient()` window (after `Object.assign(S, ...)` but before deferred render), `s.id` is already the new patient's ID — history renders correctly for the new patient

Dataset Flag Usage

- None

innerHTML Replacement Usage

- `#historyList` : full innerHTML replacement on every render call

Stale Node Risks

- **History ID scope race:** `History.load` reads `S.id` at call time. If `History.add()` is called during a cross-patient transition before `S.id` is fully applied, the history entry is logged to the wrong patient's ledger. In `selectTx()` → `History.add()`, `S.id` is the current patient. The risk applies if an async callback fires `History.add()` using a captured (but now stale) patient ID.

Orphan Listener Risks

- Event delegation on `#historyList` container — listeners survive innerHTML rebuilds
- Individual history item elements are destroyed and recreated on every rebuild

Async Overwrite Risks

- `History.render()` is always synchronous — no async DOM writes
- The 300ms deferred `render(S)` in `switchPatient()` does NOT re-call `History.render()` — history is not overwritten by the deferred render

Shared DOM Ownership

- `History.render()` is the sole writer to `#historyList`, but it is called from many orchestration points

Render Recovery Behavior

- None. No error boundary on `History.render()`.

Fallback Replacement Behavior

- None

Lazy Rendering Behavior

- None. History always renders synchronously.

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- None directly. History render is always synchronously triggered.

Visibility-Controlled Rendering

- `S.historyOpen` tracks panel open/close state but does not gate `History.render()` execution — render always runs regardless of panel visibility

Structural vs. Patch Rendering Distinction

- **Structural only.** Full innerHTML replacement on every call.
-

7. Report Button / Loading Lifecycle

DOM Ownership

Primary owner: `generateReport()` — writes `btn.classList` and `btn.innerHTML` **Async restore owner:** Anonymous `setTimeout (800ms, plain setTimeout , NOT in CleanupRegistry )`

Creation Path

- Button markup is static HTML, always present
- No creation lifecycle — button is permanently in DOM

Update Path

- **On click:** `btn.classList.add('btn-loading') + btn.innerHTML = '<i ...> Generating...'`
- **On restore (800ms timer):** `btn.classList.remove('btn-loading') + btn.innerHTML = '<i ...> Report'`

Destruction Path

- None. Button is never removed from DOM.

Rebuild Triggers

- `generateReport()` click

Conditional Rendering Behavior

- If report generation fails (falsy compound/ai result): button is restored immediately via direct DOM write inside the guard block — does NOT wait for 800ms timer
- Normal path: 800ms timer always fires regardless of report success or failure

Dataset Flag Usage

- None

innerHTML Replacement Usage

- `btn.innerHTML` : written twice — once immediately on click (loading state), once on 800ms timer (restore)

Stale Node Risks

- 800ms restore timer fires against `btn` DOM reference captured at click time
- If page navigates away within 800ms, `btn` reference is potentially stale (DOM destroyed)
- CleanupRegistry does NOT manage this timer — `pagehide` cleanup does not cancel it

Orphan Listener Risks

- The 800ms timer closure holds a reference to `btn` . If the DOM is destroyed before the timer fires, the timer callback writes to a detached DOM node — silent no-op in most browsers, potential exception in strict environments.

Async Overwrite Risks

- Double-click scenario: if `generateReport()` is called twice before the first 800ms restore fires, two overlapping timers both hold a reference to `btn` and both will write `btn.innerHTML = '<i ...>Report'` at ~800ms intervals. No corruption — same write, two timers.

Shared DOM Ownership

- `btn.innerHTML` / `btn.classList` : shared between the synchronous click handler and the orphaned async timer

Render Recovery Behavior

- Guard blocks in compound/multi/single branches restore button state on failure before return

Fallback Replacement Behavior

- On failure (falsy AI result): `btn.classList.remove('btn-loading') + btn.innerHTML = '<i ...>Report'` inline — not via timer

Lazy Rendering Behavior

- None

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- 800ms plain `setTimeout` for button restore — NOT in `CleanupRegistry`
- 1200ms `CleanupRegistry.timer()` for `URL.revokeObjectURL()` — IS in registry, will be cancelled on `pagehide`

Visibility-Controlled Rendering

- None

Structural vs. Patch Rendering Distinction

- **Patch only.** Button is never structurally rebuilt — only `innerHTML` and `classList` are mutated.
-

8. Edit Form Lifecycle

DOM Ownership

Primary owner: `buildEditForm()` — full `innerHTML` replacement of `#editFields` **State write owner:** `saveEdit()` → `setState()` / `Object.assign(S, ...)`

Creation Path

- `buildEditForm()` : full `innerHTML` replacement of `#editFields` with form controls built from current `s` values
- Called from `switchPatient()` (after `Object.assign(S, newPatient)` — reads new patient values)
- Called from `addPatient()` for new patient

Update Path

- No patch path for the form structure itself — `buildEditForm()` always does full `innerHTML` replacement
- Form field values are set by user input (native browser behavior after `buildEditForm()`)
- `updateSliderPositions()` writes to slider elements in `#whatIfContainer` (NOT in `#editFields`) — separate DOM region

Destruction Path

- Implicit: `buildEditForm()` overwrites `#editFields` on every call — prior form controls are destroyed
- Explicit visibility: `exitEdit()` → `editFields.style.display = 'none'` + `infoDisplay.style.display = 'block'`
- `switchPatient()` → `cancelEdit()` (if `S.editing`) → `exitEdit()` → then `buildEditForm()` with new patient data

Rebuild Triggers

- `switchPatient()` — always (rebuilds for new patient)
- `addPatient()` — for new patient form
- NOT rebuilt on `saveEdit()` — form persists after save

Conditional Rendering Behavior

- `S.editing` controls form visibility via `style.display`
- `buildEditForm()` is called regardless of `S.editing` state in `switchPatient()` — form is rebuilt even if not visible
- `cancelEdit()` → `exitEdit()` hides form and shows `#infoDisplay`

Dataset Flag Usage

- None on edit form elements

innerHTML Replacement Usage

- `#editFields` : full innerHTML replacement by `buildEditForm()` on every call

Stale Node Risks

- `switchPatient()` calls `buildEditForm()` with the new patient's `s` values. If `Object.assign(S, newPatient)` had not yet executed, form would read stale `s`. Current ordering: `Object.assign(S, ...)` → normalization → DOM resets → `buildEditForm()` — ordering is correct.
- `updateSliderPositions()` is called AFTER `buildEditForm()`. This ordering is correct: sliders are not in `#editFields`, so there is no DOM dependency between the two. But the implicit constraint is that `Object.assign(S, ...)` must precede both.

Orphan Listener Risks

- `buildEditForm()` generates form controls via innerHTML — any event listeners previously bound to individual `#editFields` child elements are destroyed on rebuild
- If listeners were bound directly to child elements (not via delegation on `#editFields`), they are orphaned on every `buildEditForm()` call
- Notes textarea listener uses a guard (`_notesInputBound`) to prevent re-binding — this is a separate element outside `#editFields`

Async Overwrite Risks

- None directly on the form. The notes 800ms debounce timer may write `S.notes` and call `saveState()` while the edit form is open — this does not overwrite form DOM but does trigger `renderPatientList()` 200ms later.

Shared DOM Ownership

- `#editFields` : owned exclusively by `buildEditForm()`
- `#whatIfContainer` sliders: owned by `updateSliderPositions()`, separate from edit form

Render Recovery Behavior

- None. No error boundary on `buildEditForm()` .

Fallback Replacement Behavior

- None

Lazy Rendering Behavior

- None. `buildEditForm()` always executes synchronously.

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- None directly. Notes autosave (800ms) is a separate system that touches `s.notes` and indirectly triggers sidebar re-render.

Visibility-Controlled Rendering

- `s.editing` controls display of `#editFields` vs. `#infoDisplay` via `style.display`
- `buildEditForm()` executes regardless of visibility

Structural vs. Patch Rendering Distinction

- **Structural only** for `#editFields` — full innerHTML replacement each time `buildEditForm()` is called.

9. SVG Tooth Highlighting System

DOM Ownership

Primary owner (single-site): `renderMainPanels` → `updateToothHighlight(state.tooth) + updateToothHighlight2(secondTooth || null)` **Primary owner (compound mode — first pass):** `renderMainPanels(effectiveState, ai)` → `updateToothHighlight(effectiveState.tooth)` **Primary owner (compound mode — second pass):** `render()` directly → `updateToothHighlight(primaryTooth) + updateToothHighlight2(secondaryTooth)` — OVERWRITES first pass **Preview owner:** `updatePreview()` does NOT call `updateToothHighlight` — SVG is NOT updated during preview

Creation Path

- SVG tooth elements are static markup in the HTML
- No creation — `updateToothHighlight` and `updateToothHighlight2` mutate attributes/classes on existing SVG elements

Update Path

- `updateToothHighlight(tooth)` : sets SVG fill, stroke, or class attributes on the tooth path element matching `tooth`
- `updateToothHighlight2(tooth || null)` : sets SVG attributes for the second tooth; clears prior highlight if `null`

Destruction Path

- `updateToothHighlight2(null)` : clears/resets the second tooth highlight
- `switchPatient()` → `render(S)` → highlight updated for new patient

Rebuild Triggers

- Every `render(S)` call → both `updateToothHighlight` calls (via `renderMainPanels` and/or direct in `render()`)
- `switchPatient()` → deferred `render(S)` → highlights updated for new patient

Conditional Rendering Behavior

- **Single-site path:** `updateToothHighlight(state.tooth) + updateToothHighlight2(secondTooth || null)` via `renderMainPanels`
- **Compound path (two-pass):**
 - Pass 1: inside `renderMainPanels(effectiveState, ai)` → `updateToothHighlight(effectiveState.tooth)`
 - Pass 2: `render()` directly calls `updateToothHighlight(primaryTooth) + updateToothHighlight2(secondaryTooth)` — overwrites pass 1
 - Pass 2 is the load-bearing result for compound mode

Dataset Flag Usage

- None. SVG element attributes are mutated directly.

innerHTML Replacement Usage

- None. SVG highlighting uses attribute mutations, not innerHTML.

Stale Node Risks

- `updatePreview()` does NOT call `updateToothHighlight` — SVG tooth highlight does NOT update during preview mode. If the user changes `tempState` values via sliders, the SVG reflects the last committed `S.tooth`, not any preview tooth.
- No stale node risk from innerHTML (attributes are overwritten, not appended).

Orphan Listener Risks

- SVG tooth elements are static — never rebuilt. No orphan listener risk.

Async Overwrite Risks

- None. Both SVG passes are synchronous within `render(S)` execution. The second pass overwrites the first synchronously before any async operation begins.
- The two-pass overwrite is deliberate — the first pass result is explicitly expendable in compound mode.

Shared DOM Ownership

- SVG tooth elements are written by both passes in compound mode
- If `renderMainPanels` is ever extracted independently, the second-pass overwrite from `render()` will not occur — compound SVG state will be wrong (first pass only, using `effectiveState.tooth` instead of the two-tooth compound configuration)

Render Recovery Behavior

- No error boundary on SVG highlight calls. Failures propagate through `withErrorBoundary`'s parent context if called from within one, or uncaught if called directly from `render()`.

Fallback Replacement Behavior

- None

Lazy Rendering Behavior

- None. SVG highlights always update synchronously.

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- None

Visibility-Controlled Rendering

- None. SVG highlights update regardless of panel visibility.

Structural vs. Patch Rendering Distinction

- **Patch only.** SVG elements are always mutated in place via attribute/class writes. No structural rebuild of SVG markup occurs.
-

10. Material Rendering System

DOM Ownership

Primary owner: `renderMaterial(state)` — writes `#matPrimary` and `#matAlt` via a 160ms deferred timer
Timer registry: `_matFadeTimer` (module-scoped) — managed by `clearTimeout` + `setTimeout` pattern; NOT in `CleanupRegistry`
Caller contexts: `renderMainPanels` (via `withErrorBoundary` on `matPrimary`) and `updatePreview()` (direct call)

Creation Path

- `#matPrimary` and `#matAlt` are static elements in HTML
- Content is never created — only replaced via `innerHTML`

Update Path

- `renderMaterial(state)` :
 1. `clearTimeout(_matFadeTimer)` — cancels any pending prior timer
 2. `_matFadeTimer = setTimeout(() => { ... }, 160)` — schedules new 160ms deferred write
 3. On fire: `#matPrimary.innerHTML = [material HTML] + #matAlt.innerHTML = [alt material HTML]`
- Every call to `renderMaterial()` cancels the prior pending write and schedules a fresh one

Destruction Path

- No explicit destruction. `innerHTML` overwrite constitutes implicit destruction of prior material content.

Rebuild Triggers

- Every `render(S)` call → `renderMainPanels` → `renderMaterial(state)`
- Every `updatePreview()` call → `renderMaterial(tempState)`

Conditional Rendering Behavior

- `withErrorBoundary` on `matPrimary` : if `renderMaterial()` throws synchronously (before the timer fires), the container is replaced with error fallback
- The timer callback itself is NOT inside `withErrorBoundary` — failures inside the 160ms callback are uncaught

Dataset Flag Usage

- None

innerHTML Replacement Usage

- `#matPrimary` : full innerHTML replacement inside the 160ms timer callback
- `#matAlt` : full innerHTML replacement inside the 160ms timer callback

Stale Node Risks

- **160ms window:** between `renderMaterial()` call and timer fire, `#matPrimary` and `#matAlt` still show content from the PRIOR render call. If `render(S)` is called for a new patient, `#matPrimary` shows the old patient's material for up to 160ms.
- During `switchPatient()` 300ms window: `renderMaterial()` is NOT called directly during `switchPatient()` setup. The 300ms deferred `render(S)` calls `renderMaterial()`, which then has its own 160ms timer. Material content could lag new patient display by up to 460ms (300ms defer + 160ms timer) after patient switch.

Orphan Listener Risks

- `_matFadeTimer` closure holds reference to `#matPrimary` and `#matAlt` DOM elements
- If page navigates away while timer is pending, the timer fires against detached DOM — NOT in `CleanupRegistry`, so not cancelled on `pagehide`

Async Overwrite Risks

- **Rapid render scenario:** if `render(S)` fires twice within 160ms, the first timer is cancelled by `clearTimeout(_matFadeTimer)` inside the second `renderMaterial()` call. Only the second timer fires. The 160ms window therefore always shows the PREVIOUS content, never a flash of the first render's content.
- **Preview vs. apply sequence:** `updatePreview()` calls `renderMaterial(tempState)` → 160ms timer. If `whatIfApply` fires within 160ms, `render(S)` calls `renderMaterial(state)` → cancels preview timer → schedules new 160ms timer with `s` values. Only the applied-state material renders.

Shared DOM Ownership

- `#matPrimary` and `#matAlt` : shared between `renderMaterial()` calls from `renderMainPanels` and `updatePreview()`
- Both write to the same elements via the same `_matFadeTimer` singleton
- Cancellation-via-overwrite is the coordination mechanism — last caller wins

Render Recovery Behavior

- `withErrorBoundary` on `matPrimary` : catches synchronous failures in `renderMaterial()` and replaces container with error card
- Does NOT catch failures in the 160ms timer callback

Fallback Replacement Behavior

- `withErrorBoundary catch: #matPrimary` container replaced with error card HTML

Lazy Rendering Behavior

- All material DOM writes are deferred by 160ms via `_matFadeTimer` — this is a timer-deferred render, not a lazy/observer-deferred render

Observer-Controlled Rendering

- None

Timer-Controlled Rendering

- `_matFadeTimer ( setTimeout , 160ms)`: the primary and only write path for `#matPrimary` and `#matAlt`
- Not in `CleanupRegistry` — survives `pagehide` cleanup
- Singleton — only one pending write at any time; new calls cancel the prior pending write

Visibility-Controlled Rendering

- None. Material renders regardless of scroll position.

Structural vs. Patch Rendering Distinction

- Structural only.** Full innerHTML replacement on every timer fire. No patch path.

Cross-System DOM Lifecycle Summary

Elements with Multiple Owners (Shared DOM Ownership Map)

Element	Owners	Coordination Mechanism
<code>#aiCardBody</code>	<code>buildAICardStructure , showSkeleton , withErrorBoundary catch</code>	<code>dataset.built</code> flag; last write wins
<code>#comparisonTableBody</code>	<code>renderComparisonTable , updatePreview (direct), toggleMultiSite (pre-wipe)</code>	None; last write wins
<code>#compoundSummaryBody</code>	<code>renderCompoundSummary</code>	Single writer, multiple trigger paths
<code>#patientList</code>	<code>renderPatientList</code> (called from 5+ paths)	None; last write wins
<code>#matPrimary / #matAlt</code>	<code>renderMaterial (via renderMainPanels and updatePreview)</code>	<code>_matFadeTimer</code> singleton; last caller cancels prior

SVG tooth paths	updateToothHighlight , updateToothHighlight2 (two-pass in compound)	Sequential overwrite; second pass is authoritative
#previewModeBanner	updatePreview , whatIfApply	None; not reset by switchPatient
btn (report)	generateReport sync, 800ms orphaned timer	None; orphaned timer always fires

Timer Registry (All Active Async DOM Mutators)

Timer	Variable	Duration	Registry	Cancellation	DOM Target
Typewriter animation	_typewriterTimer	30ms interval	None	Next typewriterEffect() call	recText (in #aiCardBody)
Material fade	_matFadeTimer	160ms	None	Next renderMaterial() call	#matPrimary , #matAlt
Report button restore	Anonymous	800ms	None (orphan)	Never	btn.innerHTML , btn.classList
URL revoke	(CleanupRegistry)	1200ms	Yes	pagehide	None (browser API)
State autosave	_saveStateTimer	200ms	None	Next saveState() call	#patientList (via renderPatientList)
Notes autosave	_notesDebounce	800ms	None	Next input event	S.notes , then saveState chain
Patient switch render	Anonymous	300ms	None	Never	All main panels via render (S)
Slider preview	(CleanupRegistry)	120ms	Yes	Next slider input	All preview panels
Save indicator hide	Anonymous	2000ms	None	Never	#saveIndicator

Observer Registry (All Active IntersectionObservers)

			Closure	
--	--	--	---------	--

Observer	Variable	Target	Captures	Stale Risk
Comparison table lazy	<code>_compTableObserver</code>	<code>#comparisonTableBody</code>	<code>state , ai at creation time</code>	Yes — if <code>s</code> mutates between observer creation and scroll-into-view

innerHTML Replacement Map (All Structural Rebuilds)

Element	Owner Function	Trigger Frequency	Patch Alternative
<code>#aiCardBody</code>	<code>buildAICardStructure , showSkeleton , error boundary</code>	Per patient switch + error	Yes — <code>updateAICard</code> patches when <code>dataset.built === '1'</code>
<code>#comparisonTableBody</code>	<code>renderComparisonTable</code>	Every render + every preview	None
<code>#compoundSummaryBody</code>	<code>renderCompoundSummary</code>	Every compound render	None
<code>#patientList</code>	<code>renderPatientList</code>	Every 200ms save + explicit calls	None
<code>#infoDisplay</code>	<code>renderPatientDisplay</code>	Every render	None
<code>#reasonsList</code>	<code>renderReasons</code>	Every render + every preview	None
<code>#costContainer</code>	<code>renderCost</code>	Every render + every preview	None
<code>#matPrimary / #matAlt</code>	<code>renderMaterial (timer)</code>	Every render + every preview, 160ms deferred	None
<code>#editFields</code>	<code>buildEditForm</code>	Every patient switch + add	None
<code>#historyList</code>	<code>History.render</code>	Every <code>History.add</code> + switch	None
<code>btn.innerHTML</code>	<code>generateReport</code>	Per report click + 800ms timer	N/A (single element)

Document status: Deterministic rendering preservation. No modifications, refactors, or proposals contained herein. All lifecycle behaviors reflect exact execution model as analyzed.